

Plan de Pruebas STU FRRe

Versión: 1.0

Responsable: Benjamín (QA)

Estándar de referencia: ISO/IEC 29119-3

Período de ejecución: Sprint 1 – Sprint 6

1. ALCANCE DE LAS PRUEBAS

1.1 Dentro del alcance MVP

Se cubren los siguientes módulos con pruebas **funcionales**, de **integración** y **regresión**:

Módulo	Enfoque
Visualización de horarios y mesas de examen	Filtros por carrera, comisión, materia; paginación y sincronización con API.
Modo inactivo	Timer de inactividad, activación de pantalla con noticias/alertas, retorno a home al tocar.
Mapa interactivo de la facultad	Renderizado SVG, lógica "Usted está aquí", búsqueda de aulas, zoom/pan.
Panel de Administración	CRUD de horarios, noticias, eventos, tótems; editor de widgets drag & drop; autenticación y roles.
Sistema de encuestas rápidas	Creación, visualización en tótem, envío de respuestas, persistencia.
Alertas urgentes	Publicación desde admin, visualización en tiempo real en tótem.
Soporte multi-tótem	Cada tótem muestra su configuración específica asignada desde el panel.
Modo kiosco	Comportamiento en navegador fullscreen, bloqueo de navegación, reinicio diario.

1.2 Fuera del alcance MVP

- Pruebas de seguridad ofensivas:** Se validan controles básicos (CORS, auth, roles).

- **Pruebas de accesibilidad WCAG completas** (solo se verifica navegación por teclado y contraste básico).
- **Traducción / i18n** (no está contemplado en MVP).

2. CONFIGURACIÓN Y FLUJO DEL EQUIPO

2.1 Responsabilidades por rol

Rol	Entrega a QA	Formato
Agustín	Endpoints REST funcionales + colección Postman/BRuno + migraciones DB.	Incremento deployado en dev con docker-compose.
Tomás	Componentes React con datos mock primero, luego migrados a API real.	PR a rama develop + build de Storybook.
Benjamín (QA)	Reporte de bugs + resultados de pruebas (unitarias, integración, E2E).	Issues en GitHub Projects + informe semanal.

2.2 Flujo ágil de QA

1. **Inicio de iteración:** QA revisa los **criterios de aceptación** de las historias planificadas para el sprint. Si algo es ambiguo, se aclara con el PO/equipo antes de comenzar.
2. **Durante el desarrollo:** QA escribe/prerredacta los **casos de prueba en Gherkin** mientras devs codifican.
3. **Al finalizar un incremento:** Dev marca como "**Listo para QA**" en el tablero. QA valida en develop o en entorno de preview.
4. **Si se encuentra un bug:** Se crea issue con evidencia (log, screenshot, video) y se reasigna al responsable. El ítem vuelve a "**En progreso**".
5. **Cierre de historia:** Una vez superadas las pruebas, QA mueve a "**Hecho**".

3. CRITERIOS DE ENTRADA Y SALIDA

3.1 Criterios de entrada Definition of Ready – DoR

Para que una historia de usuario o incremento pase a QA, debe cumplir todos estos puntos:

- Los **criterios de aceptación** están definidos y no ambiguos.
- Los **tests unitarios del backend** pasan (pytest green).
- Los **tests unitarios del frontend** pasan (vitest / jest green).
- La **build de CI/CD** es verde (linter + typecheck + tests).
- La **migración de base de datos** es hacia adelante y hacia atrás.
- **Swagger/OpenAPI** está actualizado si se agregó/modificó un endpoint.
- No hay **console.error ni warnings** en el frontend en modo desarrollo.
- Se proporciona un **entorno accesible** (Instrucciones).

3.2 Criterios de salida Definition of Done – DoD

Se considera que un incremento está listo para cerrar cuando:

- Todos los **casos de prueba** planificados para esa historia se ejecutaron con resultado PASS.
- No hay bugs **Critical** ni **High** abiertos contra el incremento.
- Los bugs **Medium** y **Low** están documentados y aceptados por el PO (con acuerdo de resolver en sprint siguiente si aplica).
- La **cobertura de pruebas unitarias** del backend es ≥ 70 % (líneas nuevas).
- La **cobertura de pruebas unitarias** del frontend es ≥ 50 % (componentes nuevos).
- Las **pruebas E2E** de los flujos críticos pasan (Cypress/Playwright).
- La **documentación de usuario final** está actualizada si corresponde.

4. MATRIZ DE RIESGOS DE CALIDAD

Funcionalidad	Riesgo 1-5	Estrategia de prueba	Prioridad
Autenticación y roles del Panel Admin	5	Automatizada (unit + integración + seguridad básica)	 Crítica
Filtrado de horarios y mesas de examen	4	Automatizada (unit + integración API) + Manual en front	 Alta
Modo inactivo (protector de pantalla)	4	Manual (comportamiento temporal) + E2E	 Alta
Mapa interactivo SVG + "Usted está aquí"	3	Manual (verificación visual + arrastre) + E2E	 Media
CRUD de noticias y eventos (Admin)	3	Automatizada (API CRUD) + Manual en panel	 Media
Editor drag & drop de widgets	3	Manual (interacción compleja)	 Media
Encuestas rápidas desde el tótem	2	Automatizada (API) + E2E	 Baja
Alertas en tiempo real	3	Manual + Integración (verificar refresh/polling)	 Media

Soporte multi-tótem (config asignada)	3	Integración (verificar que Tótem A ≠ Tótem B)	● Media
Modo kiosco (fullscreen, bloqueo)	3	Manual + E2E básico	● Alta
Regresión general antes de release	–	Automatizada (regresión vía API + E2E smoke)	● Alta

Nota: Se ejecutará **regresión completa** al final de los Sprints 3 y 6 (hito de release).

5. CASOS DE PRUEBA CRÍTICOS Formato Gherkin

CP-01: Filtrado de horarios por carrera y comisión

- **DADO** que el tótem está en la pantalla de inicio
- **Y** existen horarios cargados para "Ingeniería en Sistemas" comisión "A"
- **Y** existen horarios cargados para "Ingeniería Civil" comisión "Única"
- **CUANDO** el usuario selecciona "Ingeniería en Sistemas" en el filtro de carrera
- **Y** luego selecciona comisión "A"
- **ENTONCES** se muestran únicamente los horarios correspondientes a "Ingeniería en Sistemas - Comisión A"
- **Y NO** se muestran horarios de otras carreras
- **Y** el título de la pantalla indica "Horarios - Ing. en Sistemas (Comisión A)"
- *Criterio adicional:* Si no hay horarios para la combinación seleccionada, se debe mostrar el mensaje "No hay horarios disponibles para esta selección" y un botón para limpiar filtros.

CP-02: Salto al protector de pantalla por inactividad

- **DADO** que el tótem se encuentra en la pantalla de horarios
- **Y** el tiempo de inactividad está configurado a 30 segundos
- **CUANDO** transcurren 30 segundos sin que el usuario interactúe con la pantalla (no hay clicks, ni toques, ni movimiento del mouse)
- **ENTONCES** el tótem activa el modo protector de pantalla
- **Y** se oculta la pantalla de horarios
- **Y** se muestra una presentación con noticias y alertas visibles
- **Y** el reloj en la esquina superior derecha indica la hora actual
- **Y** al hacer click/tap en cualquier parte de la pantalla
- **ENTONCES** el protector de pantalla se desactiva
- **Y** se retorna exactamente a la pantalla de horarios que estaba visible antes de la inactividad
- *Criterio adicional:* El timer debe reiniciarse si el usuario interactúa ANTES de que venzan los 30 segundos.

CP-03: Bloqueo de acceso no autorizado al Panel de Administración

- **DADO** que un usuario NO autenticado intenta acceder a la URL "/admin"
- **CUANDO** el navegador solicita la ruta "/admin"
- **ENTONCES** el servidor responde con código HTTP 401 (Unauthorized)

- **Y** se redirige automáticamente a `/admin/login`
- **Y** no se renderiza ningún contenido del panel
- **DADO** que un usuario con rol "Lector" (solo lectura) intenta acceder a `/admin/horarios/create`
- **CUANDO** envía una solicitud POST a `/api/horarios/` con datos válidos
- **ENTONCES** el servidor responde con código HTTP 403 (Forbidden)
- **Y** el mensaje de error indica "No tienes permisos para realizar esta acción"
- **Y** el horario no es creado en la base de datos
- **DADO** que un usuario con rol "Administrador" intenta acceder a `/admin/horarios/create`
- **CUANDO** envía una solicitud POST a `/api/horarios/` con datos válidos
- **ENTONCES** el servidor responde con código HTTP 201 (Created)
- **Y** el horario queda persistido en la base de datos
- **Y** se muestra en la tabla de horarios del panel

ANEXO – Herramientas propuestas

Tipo	Herramienta	Justificación
Unit tests backend	pytest + pytest-django	Estándar en el ecosistema Django.
Unit tests frontend	vitest + @testing-library/react	Rápido, nativo en Vite.
E2E	Playwright	Soporte multi-browser, buen manejo de async, graba video.
API testing	requests (pytest) + colección en Bruno	Ligero, versionable en repo.
Coverage backend	pytest-cov	Reporte integrado con CI.
Coverage frontend	vitest --coverage	Reporte en lcov.
CI/CD	GitHub Actions	Ya integrado con el repo del proyecto.